

Node.js Coding Standard — v2.0

Owner: Raja - **Approved:** Praveen (CTO)
Applies to: all Node services, Express and NestJS
Supersedes: Node.js Coding Standard v1.0

Scope and status

This standard applies to **new and changed code**. Existing code is not retrofitted — a rule arriving here does not create a backlog of rewrites. When you touch a file, the code you write in it follows this document.

Where a rule has sample code, that code is the house pattern. Copy it rather than writing your own version.

Sections 1–12 mirror the Angular and Next.js standards section for section. Sections 13 and 14 are backend-specific.

Version policy

Node runs the current LTS. Upgrades are planned work, scheduled at least once per LTS cycle, not deferred until something forces them. Skipping upgrades for a year turns a routine bump into a migration and leaves known vulnerabilities in place for that whole period.

Packages are chosen at their **stable** release, not the newest. The newest version carries the bugs nobody has hit yet, and being first to find them is not a benefit to a production platform.

Enforcement tags

Tag	Meaning
[LINTER]	A tool in the repo catches it — ESLint, <code>tsc</code> .
[CI]	A pipeline step or Git-host setting catches it — Gitleaks, Trivy, coverage gate, branch protection.
[NEITHER]	A human must judge it in review.
[LINTER + NEITHER]	The linter covers part; review covers the rest.
[CI + NEITHER]	The pipeline covers part; review covers the rest.

Check: notes name the specific tool, or explain why a rule is not automatable. A **backstop** is a narrower rule a tool *can* enforce, used where the real rule cannot be automated.

Roughly half of this document is **[NEITHER]** — and in a backend that half is where the money is. Authorisation, idempotency, transaction boundaries and amount handling are all human-judged. That is the measured reason review is mandatory on every PR.

1. Naming

- [LINTER]** Files and folders kebab-case; classes and types PascalCase; variables and functions camelCase; constants and env keys UPPER_SNAKE_CASE. *Reason:* mixed conventions make grep and file search unreliable across a codebase this size. **Check:** `eslint-plugin-check-file`, `@typescript-eslint/naming-convention`.
- [LINTER]** Files carry a role suffix: `.controller.ts`, `.service.ts`, `.repository.ts`, `.dto.ts`, `.test.ts`. *Reason:* a reviewer should know a file's responsibility from the tree alone. **Check:** `eslint-plugin-check-file`.
- [NEITHER]** Names state what a thing is, not how it was built: `getActiveLoans()`, never `getData2()` or `helperFn`. *Reason:* juniors inherit these names for years.
- [LINTER + NEITHER]** Booleans read as assertions — `isActive`, `hasExpired`, `canWithdraw`. **Check:** Linter — `@typescript-eslint/naming-convention`. Human — names that pass the rule but still say nothing.
- [NEITHER]** Methods that return a nullable result start with `find`; methods that throw when the record is missing start with `get`. *Reason:* the caller then knows the failure mode from the call site, without opening the implementation.

1.1 Wire format across stacks

[NEITHER] JSON payloads are camelCase on the wire, in both directions.

Our Python services use `snake_case` internally and Node uses `camelCase`. Both are correct in their own language. The contract between them is camelCase.

Node does not convert: our DTOs are camelCase and that is what goes on the wire. Python converts at its boundary with Pydantic field aliases.

If one of our endpoints emits `snake_case`, that is our defect, not the frontend's problem to map around.

2. Project structure

Feature-first, not layer-first. A feature owns its route, logic, data access, validation and types.

```
src/
  modules/
    loans/
      loans.controller.ts    # HTTP only — no business logic
      loans.service.ts       # business logic
      loans.repository.ts    # all DB access for this module
      loans.dto.ts           # request + response shapes and validation
      loans.test.ts
    common/
      middleware/ guards/ errors/ utils/
    config/                  # env parsing + validation, one place
    db/                     # connection, shared models, migrations
    jobs/                   # cron, queues, workers
    observability/          # logger, correlation id, alerting
  docs/
  test/                    # integration / e2e
  .env.example
```

- **[NEITHER]** **Feature-first on every new project. No exceptions.** The same tree for Express and NestJS — Nest wires it through modules, Express wires it manually; structure does not change with framework. Existing projects on another layout are not migrated.
- **[NEITHER]** **Controllers do HTTP. Services decide. Repositories touch the database.** A controller that queries the database directly, or a service that reads `req`, has crossed a boundary that exists to make the logic testable.
- **[LINTER]** No cross-module imports of internals. Modules talk through an exported service, never another module's repository. **Check:** `eslint-plugin-boundaries`.

3. Error handling, validation & API conventions

- **[NEITHER]** **Every external input is validated at the boundary with a schema** — body, params, query, headers, webhook payloads. *Reason:* unvalidated input is the single most common source of production incidents in this stack.
- **[NEITHER]** One global error handler. Handlers and services throw typed errors and never send responses themselves. *Reason:* otherwise error shape drifts per endpoint.
- **[LINTER]** No empty `catch`, and no `catch` that only logs and continues. *Reason:* silent failure turns a five-minute bug into a two-day investigation. **Check:** `no-empty`, plus review for the log-and-continue case.
- **[LINTER]** Every async call is awaited or has an attached rejection handler. No floating promises. *Reason:* unhandled rejections terminate the process on modern Node. **Check:** `@typescript-eslint/no-floating-promises`.
- **[NEITHER]** Internal error detail, stack traces and driver messages are never returned to the client.
- **[NEITHER]** Breaking API changes ship under a new version path. Existing fields are not removed or retyped in place — we do not control all consumers.

3.1 The error contract

Every service returns the same error body. The frontends parse it once — see their section 3.1.

```
// src/common/errors/app-error.ts

export class AppError extends Error {
  constructor(
    readonly code: string,          // machine-readable: 'LOAN_ALREADY_SETTLED'
    readonly status: number,        // HTTP status
    message: string,               // developer-facing; never shown to a user
    readonly details?: Record<string, string[]>,
    readonly cause?: unknown,
  ) {
    super(message);
    this.name = 'AppError';
  }
}

export const NotFound = (code: string, msg: string) => new AppError(code, 404, msg);
export const Conflict = (code: string, msg: string) => new AppError(code, 409, msg);
export const Forbidden = (code = 'FORBIDDEN', msg = 'Not permitted') => new AppError(code, 403, msg);
export const BadRequest = (code: string, msg: string, details?: Record<string, string[]>) =>
  new AppError(code, 400, msg, details);
```

Two kinds of invalid deserve two different statuses. A malformed request is 400. A well-formed request that conflicts with current state — settling an already-settled loan — is 409. Returning 400 for both means the frontend cannot tell a bug from a race.

3.2 The global error handler

```
// src/common/errors/error-handler.ts
import type { ErrorRequestHandler } from 'express';

export const errorHandler: ErrorRequestHandler = (err, req, res, _next) => {
  const correlationId = correlationStore.get();

  if (err instanceof ZodError) {
    const details = err.flatten().fieldErrors as Record<string, string[]>;
    return res.status(400).json({ code: 'VALIDATION_FAILED', message: 'Invalid request', details });
  }

  if (err instanceof AppError) {
    // 4xx is the caller's problem and is not an alert.
    if (err.status >= 500) logger.error(err.message, { code: err.code, cause: err.cause });
    else logger.warn(err.message, { code: err.code, path: req.path });

    return res.status(err.status).json({ code: err.code, message: err.message, details: err.details });
  }

  // Anything unrecognised is ours, and someone needs to look at it.
  logger.error('Unhandled error', { err, path: req.path, method: req.method });
  alert.fire('UNHANDLED_EXCEPTION', { path: req.path, correlationId });

  return res.status(500).json({
    code: 'INTERNAL_ERROR',
    message: 'An unexpected error occurred', // never the real message
    correlationId,
  });
};
```

Return the correlation id on a 500. It costs nothing and it is what turns a support ticket into a log lookup. The frontends display it on their error screens.

In NestJS the same logic lives in a global `ExceptionHandler`. The rules are identical.

3.3 Validation at the boundary

```
// src/modules/loans/loans.dto.ts
import { z } from 'zod';

export const createLoanSchema = z.object({
  borrowerId: z.string().uuid(),
  principalMinor: z.string().regex(/^\d+$/), // money as a string – section 12
  termDays: z.number().int().positive().max(3650),
});
```

```
interestBps: z.number().int().min(0).max(10_000),
});

export type CreateLoanDto = z.infer<typeof createLoanSchema>;
```

```
// src/common/middleware/validate.ts
export const validate = (schema: ZodSchema, source: 'body' | 'query' | 'params' = 'body') =>
  (req: Request, _res: Response, next: NextFunction) => {
    const parsed = schema.safeParse(req[source]);
    if (!parsed.success) return next(parsed.error);
    req[source] = parsed.data; // downstream code sees the parsed, typed value
    next();
  };
```

`z.infer` means the schema is the single definition. A hand-written interface alongside a schema will drift.

3.4 Response envelope

- [NEITHER] Success responses carry the resource directly, or `{ items, total, page }` for a list. Errors carry `{ code, message, details? }`.
- [NEITHER] The HTTP status carries the category; `code` carries the specific case. **Clients cannot branch reliably on prose messages.**
- [NEITHER] Never HTTP 200 with `{ success: false }`. Status codes exist for this.

4. Logging, observability & alerting

4.1 Log levels

The same four meanings across every stack in the company.

Level	Meaning
error	A human needs to look at this.
warn	Degraded but handled.
info	A state change worth auditing.
debug	Off in production.

One `info` line per database call means nobody reads `info` at all.

4.2 Correlation IDs

[NEITHER] Every request carries a correlation id through every log line, downstream call and response header.

Reason: without it, tracing one user's request across services is guesswork. The frontends generate the id and send it as `X-Correlation-Id`; we honour it when present and mint one when absent.

```
// src/observability/correlation.ts
import { AsyncLocalStorage } from 'node:async_hooks';
import { randomUUID } from 'node:crypto';

const als = new AsyncLocalStorage<{ correlationId: string }>();

export const correlationStore = {
  get: () => als.getStore()?.correlationId ?? null,
};

export const correlationMiddleware: RequestHandler = (req, res, next) => {
  const correlationId = (req.header('X-Correlation-Id') ?? randomUUID()).slice(0, 64);
  res.setHeader('X-Correlation-Id', correlationId);
  als.run({ correlationId }, () => next());
};
```

[NEITHER] **Outbound calls to our own services forward the header.** A chain that drops it at the second hop is no more useful than having none.

4.3 Structured logs with redaction enforced in code

[NEITHER] **Never logged, in any environment:** passwords, tokens, API keys, private keys, seed phrases, OTPs, **Authorization** headers, card or bank data, full PII, KYC documents, raw webhook secrets.

Reason: logs are copied, shipped and retained far beyond their intended audience. A denylist in a document is a hope; a denylist in the logger is a control.

```
// src/observability/logger.ts
import pino from 'pino';

export const logger = pino({
  level: process.env.LOG_LEVEL ?? 'info',
  redact: {
    paths: [
      'req.headers.authorization', 'req.headers.cookie',
      '*.password', '*.passwd', '*.token', '*.accessToken', '*.refreshToken',
      '*.apiKey', '*.api_key', '*.secret', '*.privateKey', '*.mnemonic', '*.seed',
      '*.otp', '*.pin', '*.cvv', '*.cardNumber', '*.aadhaar', '*.pan', '*.ssn',
      '*.email', '*.phone',
    ],
    censor: '[REDACTED]',
  },
  formatters: { level: (label) => ({ level: label }) },
  mixin: () => ({ correlationId: correlationStore.get() }), // on every line, free
});
```

If something sensitive does reach a log, treat it as a leaked credential and rotate it. The log has already been shipped and retained. Removing the log line changes nothing.

[LINTER] No `console.log` in committed code outside scripts. *Reason:* it bypasses levels, redaction and log shipping. **Check:** `no-console`, with `scripts/**` exempted.

4.4 Alerting — a log file is not an alert

[NEITHER] **Every service has an error alerting system. Writing to a log file is not sufficient.**

Nobody reads log files daily, so an error that only reaches a file is an error nobody knows about until a client reports it. This rule is the reason this section exists at all.

Minimum events that must raise an alert:

Event	Why
Unhandled exception, process crash	The service is degraded right now
Failed background or cron job	Silent, and compounds daily
Failed third-party API or webhook processing	Our data is now out of sync with theirs
Failed on-chain transaction, event listener stopped	Money in flight, and the listener does not restart itself
Database connection loss	Everything downstream is about to fail
Authentication or payment failures above a threshold	Either an outage or an attack

Alerts carry enough context to act on: environment, service, correlation id, error code and a short message. *An alert that only says “Error” costs more time than it saves.*

```
// src/observability/alert.ts
import * as Sentry from '@sentry/node';

const WINDOW_MS = 5 * 60_000;
const seen = new Map<string, number>();

/** Rate-limited so a repeating failure does not flood the channel and get muted. */
function shouldSend(key: string): boolean {
  const now = Date.now();
```

```

const last = seen.get(key);
if (last && now - last < WINDOW_MS) return false;
seen.set(key, now);
return true;
}

export const alert = {
  fire(event: AlertEvent, context: Record<string, unknown> = {}): void {
    const payload = {
      event,
      environment: config.env,           // channels are separated by environment
      service: config.serviceName,
      correlationId: correlationStore.get(),
      ...context,
    };

    logger.error({ alert: payload }, `ALERT ${event}`);
    Sentry.captureMessage(`ALERT ${event}`, { level: 'error', extra: payload });

    if (!shouldSend(`${event}:${JSON.stringify(context.code ?? '')}`)) return;
    void notifyChannel(payload);          // Teams / Slack webhook for this environment
  },
};

```

```

// src/main.ts – the two handlers every service needs
process.on('unhandledRejection', (reason) => {
  logger.error({ reason }, 'Unhandled rejection');
  alert.fire('UNHANDLED_REJECTION', { reason: String(reason) });
});

process.on('uncaughtException', (err) => {
  logger.error({ err }, 'Uncaught exception');
  alert.fire('UNCAUGHT_EXCEPTION', { message: err.message });
  // Flush, then exit. A process in an unknown state does not keep serving traffic.
  void Sentry.close(2000).then(() => process.exit(1));
});

```

- **[NEITHER] Alert channels are separated by environment.** Production noise is never mixed with dev or staging, or production alerts get muted along with the noise.
- **[NEITHER]** Alerts are deduplicated and rate-limited, as above.
- **[NEITHER] Production minimum:** a health endpoint, error alerting as described, and uptime monitoring.

5. Testing expectations

Testing here means what the developer owes **before** a PR is raised. QA validates the feature; the developer is responsible for proving the code works before it reaches them.

- **[NEITHER] The developer tests their own work before raising the PR.** A PR is not a request for someone else to find out whether it runs — sending untested code to review or QA moves the cost of a defect to the most expensive point.
- **[NEITHER] Unit tests are required for:** money, amount, interest and fee calculations; date and schedule logic; state-transition rules; shared utilities. *Reason:* these produce a wrong answer silently instead of failing, so no manual tester will catch them.
- **[NEITHER]** Every endpoint is verified by the developer for its success path, its auth-failure path, and its validation-failure path before handover.
- **[NEITHER]** Failure cases are tested against realistic data, not the happy path on one clean record: invalid input, missing fields, expired session, third-party down, insufficient balance, duplicate request. *Reason:* most production defects are unhandled failure paths on existing messy data, not broken success paths.
- **[CI]** Tests never run against live third-party APIs, mainnet, or production data. *Reason:* a test that depends on an external system fails randomly, then gets ignored, then gets deleted.
- **[CI]** Existing tests pass before merge. A developer does not disable, skip or delete a failing test to get a PR through. If a test is wrong, it is fixed and the reason goes in the PR.
- **[NEITHER]** Every fixed bug ships with a test that fails without the fix.
- **[CI]** New and changed lines meet the project's coverage threshold; overall coverage must not drop in a PR. **Check:** Jest/Vitest thresholds plus a diff-coverage step. *A percentage target on legacy code is meaningless; a ratchet on new code is not.*
- **[NEITHER]** Repository tests run against the **same database engine as production**, through Testcontainers. An in-memory substitute accepts queries the real engine rejects, so a green test proves nothing.

6. PR & merge rules

- [CI] **One approval minimum, from someone other than the author, on every PR in every project.** Two approvals when the change touches authentication, payments, migrations, or `common/`. **Check:** branch protection plus CODEOWNERS.
- [CI] Blocks merge: failing CI, lint or build errors, self-approval, protected-branch bypass, unresolved blocking comments, dropped coverage, or a migration with no rollback plan.
- [NEITHER] **Target 400 changed lines**, excluding lock files, generated code and pure renames. Larger PRs carry a note saying why and how to review them. *Reason:* past roughly that point a reviewer skims instead of reads, and approval stops meaning anything.
- [LINTER] Branches `feature/<ticket>-<slug>`, `fix/<ticket>-<slug>`. Commits describe the change in plain English, not “fix”, “update”, “changes”.
- [CI + NEITHER] **Required PR description fields, all of them:** what changed · why it changed · how it was tested · security/architecture impact · breaking changes · related ticket · deployment requirements.
 - *How it was tested* states what was run and what was checked. **“Tested locally” is not an answer.**
 - *Deployment requirements* lists everything the deploy needs beyond the code: new or changed environment variables, migrations or seed data, scripts to run before or after, config or infrastructure changes, and the order they run in. **If nothing is needed, write “None”** — silence is not the same as nothing, and a deploy that fails because one variable was never mentioned costs far more than the line it takes to write it.
- [CI] Gitleaks and a dependency scan run on every PR. **A detected secret blocks the merge and triggers rotation of that credential**, not just its removal from the diff — once committed, it is in the history and must be treated as leaked.
- [CI] The dependency scan blocks on High and Critical findings and warns on Medium. Existing findings are baselined as a remediation backlog so CI blocks on new problems, not old ones. **The baseline is not regenerated to make a failing pipeline pass.**

On the review requirement. About half the rules in this document are [NEITHER], and in a backend that half is authorisation, idempotency, transaction boundaries and money. On a project where review is switched off, none of that is checked by anything. That is why the rule is per-company and not per-project. Exceptions are granted by the CTO for a named person — a sole developer on a service — not chosen per project.

7. Review checklist

1. [NEITHER] Inputs validated at the boundary, with a schema?
2. [NEITHER] Errors handled, typed, and surfaced in the standard shape?
3. [NEITHER] **Authorisation checked, not just authentication — can user A act on user B's resource?** (8.3)
4. [LINTER + NEITHER] No secrets, keys or PII in code, logs, tests or fixtures?
5. [NEITHER] Queries indexed, bounded, and free of N+1?
6. [NEITHER] **Money, decimals and units correct; no float arithmetic on amounts?** (section 12)
7. [NEITHER] Tests present for the logic changed, including failure paths?
8. [LINTER] No dead code, commented-out blocks, stray `console.log`, or leftover debug flags?
9. [LINTER + NEITHER] Naming and structure match sections 1 and 2?
10. [NEITHER] External and on-chain calls have timeout, retry policy, and idempotency where retried?
11. [NEITHER] Backward compatibility of API and DB schema considered?
12. [NEITHER] Multi-step writes inside a transaction? (13.2)
13. [NEITHER] `.env.example`, README, migrations and deployment steps updated and declared?
14. [NEITHER] Does this change need a new alert, and is it wired? (4.4)
15. [NEITHER] Could the newest developer on the team read this without asking the author?

8. Security & secrets

8.1 Secrets

- [CI] Secrets live only in environment variables or the secret manager. Never in source, config files, comments, tests or commit history. *Reason:* a

leaked key in git history is a rotation event, not a revert. **Check:** Gitleaks, blocking.

- [NEITHER] **Secrets come from the environment with no default fallback.** A default gets shipped, and then gets forgotten.

```
// ✖ this will reach production and nobody will notice
const jwtSecret = process.env.JWT_SECRET ?? 'dev-secret';

// ✔ the service refuses to start
const jwtSecret = requireEnv('JWT_SECRET');
```

- [NEITHER] `.env.example` lists every required key with a dummy value and no real ones. New developers should not have to ask which variables exist.

AWS: production secrets come from Secrets Manager or Parameter Store, read at startup through the config module. Rotation is a Secrets Manager concern, not a redeploy.

8.2 Authentication

- [NEITHER] Authentication on every non-public route.
- [NEITHER] Passwords hashed with bcrypt or argon2 — never encrypted, reversible, or logged. Comparison uses the library's own compare function. *Reason:* reversible storage means one database leak exposes every account.
- [NEITHER] **Everything issued for verification expires:** sessions, access and refresh tokens, OTPs, password-reset and email-verification links, signed URLs. Logout, password change or role change invalidates existing sessions. *Reason:* a credential that outlives the access it was granted for means revoking access does nothing.
- [NEITHER] **Sessions are set as httpOnly, Secure, SameSite cookies** by this service. The frontends never store a token in `localStorage`; our cookie is what makes that possible.
- [NEITHER] Refresh tokens rotate on use, and the **old token is invalidated**. A reused refresh token is treated as replay: revoke the session and alert.

8.3 Authorisation — the ownership check

[NEITHER] **Authentication alone does not stop user A reading user B's data.** Every endpoint that touches a record checks ownership or role, in the service, every time.

This is **IDOR**, and it is the most likely real vulnerability in our systems. A valid token for user A, with user B's id in the URL, returning user B's record. It needs no tools — a user changes a number in the address bar and finds it by accident. Scanners miss it: the request is well-formed, authenticated, and returns 200.

The fix is in how you query, not in a check you remember to add:

```
// ✖ returns anyone's loan to anyone who is signed in
const loan = await loanRepository.findById(id);

// ✔ the query cannot return another user's record
const loan = await loanRepository.findByIdForOwner(id, currentUser.id);
if (!loan) throw NotFound('LOAN_NOT_FOUND', 'Loan not found');
```

Scope the query to the caller and there is no check to forget.

- [NEITHER] **Return 404, not 403, for a record the caller does not own.** A 403 confirms the record exists, which is itself a disclosure.
- [NEITHER] Role checks are additional, not a substitute. An admin role still has an explicit check.
- [NEITHER] **Every PR that adds an endpoint names, in its description, how ownership is enforced on it.**

8.4 Request safety

- [NEITHER] Every inbound webhook or callback **verifies its signature or HMAC before the payload is processed.** *Reason:* the URL is not a credential, and an unverified endpoint is an unauthenticated write path into the system.
- [NEITHER] Rate limiting on auth, OTP, and any endpoint that costs money or gas.
- [LINTER + NEITHER] Security headers, a CORS allowlist — never `*` with credentials — and payload size limits enabled in production.
- [NEITHER] Parameter binding only. Never build SQL or JPQL by string concatenation.
- [NEITHER] File uploads restricted by size and by **actual content type**, not the filename or the client-declared MIME type. Stored privately, served through expiring signed URLs. *Reason:* unbounded or unchecked uploads are both a denial-of-service path and a way to store hostile content on our

infrastructure.

8.5 Never trust the client

[NEITHER] Client-supplied identity, amount, price, role, status or transaction hash is re-derived or re-verified server-side. **Always.**

The client controls everything it sends. A field that the server does not verify is a field the server has delegated to an attacker.

9. Dependencies & configuration

- [NEITHER] Adding a dependency needs a one-line justification in the PR: what it does, why the standard library or an existing package cannot, its last release date. *Reason:* every package is permanent attack surface and future upgrade work.
- [CI] Exact versions in `package.json`, lock file always committed, one package manager per repo. *Reason:* “works on my machine” is nearly always a lock file problem. **Check:** `npm ci` fails on a missing or stale lock file.
- [NEITHER] Unmaintained packages — no release or security fix in roughly two years — are flagged for replacement rather than adopted.
- [NEITHER] Versions are reviewed and updated as planned work, not left until something forces it. Skipping updates for a year turns a routine upgrade into a large, risky migration.
- [LINTER] **All config is read and validated once at startup through `config/`.** The app fails fast on a missing or malformed variable. *Reason:* better to fail at boot than at 2 a.m. on a code path nobody exercised. **Check:** `no-restricted-properties` on `process.env` outside `src/config/**`.

```
// src/config/index.ts
const schema = z.object({
  NODE_ENV: z.enum(['development', 'test', 'staging', 'production']),
  PORT: z.coerce.number().int().positive(),
  DATABASE_URL: z.string().url(),
  JWT_SECRET: z.string().min(32),
  ALERT_WEBHOOK_URL: z.string().url(),
});

const parsed = schema.safeParse(process.env);
if (!parsed.success) {
  console.error('Invalid configuration:', parsed.error.flatten().fieldErrors);
  process.exit(1); // fail at boot, loudly
}

export const config = parsed.data;
```

10. Maintainability & complexity

- [LINTER] Functions stay small enough to read without scrolling (~75 lines) and nesting stays shallow (3 levels). *Reason:* deep nesting is where edge cases hide. **Check:** `max-lines-per-function`, `max-depth`.
- [NEITHER] **The third repetition of the same logic gets extracted; the second does not.** Premature abstraction costs more than duplication.
- [NEITHER] Comments explain **why**, never what. Complex business rules — interest, schedules, fees, state transitions — carry a short rationale comment.
- [NEITHER] Public and shared functions and all API endpoints are documented. Every README covers setup, environment variables, and how to run locally and run tests.
- [LINTER] **No blocking synchronous calls on the request path** — sync `fs`, sync crypto, long loops. *Reason:* Node has one thread and one slow handler degrades everything. **Check:** `no-sync`.
- [NEITHER] **Performance rules:** no queries inside loops; no unbounded `find()` — paginate; select only the fields needed; index every field used for filtering or sorting; cap payload sizes; move CPU-heavy work off the request path into a job or worker; stream large files rather than loading them into memory.

11. What we do NOT do

- [NEITHER] **No floats for money, token amounts or interest.** See section 12.

- [NEITHER] No business logic in controllers or route handlers. It cannot be tested or reused.
- [LINTER] No `any` to get past the compiler. If the type is genuinely unknown, use `unknown` and narrow. **Check:** `@typescript-eslint/no-explicit-any`.
- [LINTER] No swallowed errors — empty catches, `.catch(() => {})`, or a log with no rethrow and no handling. **Check:** `no-empty`.
- [NEITHER] No sequential `await` in a loop for independent operations. Batch or parallelise with a bounded concurrency limit.
- [NEITHER] **No non-idempotent webhook and event handlers.** Providers and chains redeliver. Handling the same event twice must not create two records or move a state twice.
- [NEITHER] No trusting client-supplied identity, amount, price, role or status. Re-derive server-side, always.
- [NEITHER] **No silently retried write operations without an idempotency key.** A retried transfer is not a harmless retry.
- [LINTER] No `console.log` debugging left in, no commented-out code blocks, no dead files kept “just in case”. Git remembers.
- [NEITHER] No direct schema or data edits in production without a migration or a reviewed, dry-run-capable script.
- [NEITHER] **No multi-step database writes without a transaction.** See 13.2.
- [NEITHER] No unindexed fields on queries that filter, sort or join at scale — checked before the feature ships, not after it is slow in production.
- [NEITHER] No mixing of concerns in a single `utils` dump file. If it has thirty unrelated exports, it is not a utility.

12. Money and numeric values

[NEITHER] **Never use JavaScript `number` arithmetic on a monetary or token amount.**

`0.1 + 0.2` is `0.30000000000000004`. Floating-point rounding on financial values produces defects that are found by clients, not by us.

Kind	Type	Why
Fiat currency	<code>bigint</code> in minor units, serialised as a string	<code>number</code> loses precision above 2^{53} and on fractions
Token / on-chain	<code>bigint</code> in base units (wei)	Exact, and matches what the chain returns
Rates and percentages	integer basis points (<code>interestBps: 250 = 2.5%</code>)	No decimal to round

```
// src/common/money/money.ts

/** Minor units. 1050n = ₹10.50. Never a float, never a Number. */
export type Minor = bigint;

export function applyBps(amount: Minor, bps: number): Minor {
  if (!Number.isInteger(bps)) throw new Error('bps must be an integer');
  return (amount * BigInt(bps)) / 10_000n; // integer division, truncates
}

/** JSON cannot carry a bigint. Serialise as a decimal string. */
export const toWire = (amount: Minor): string => amount.toString();
export const fromWire = (raw: string): Minor => {
  if (!/^\d+$/.test(raw)) throw BadRequest('INVALID_AMOUNT', 'Amount must be an integer string');
  return BigInt(raw);
};
```

- [NEITHER] **Amounts cross the wire as strings**, and the frontends treat them as strings. `JSON.stringify` cannot serialise a `bigint`, and `JSON.parse` on a large number loses precision silently.
- [NEITHER] Database columns for money are `NUMERIC` / `DECIMAL` or an integer type, **never float or double**.
- [NEITHER] **Rounding is decided once, written down, and applied in one place.** Which direction, at which step, and who absorbs the remainder. Two developers rounding independently is how a ledger stops balancing.
- [NEITHER] The backend is authoritative for every total, fee and interest figure. The frontend displays what we give it.

Never:

```
const total = parseFloat(a) + parseFloat(b); // ✗
const fee = amount * 0.025; // ✗
const paise = Math.round(rupees * 100); // ✗ rounds a float that is already wrong
```

13. Data access & transactions

13.1 One session per request

[NEITHER] A request uses one database connection from the pool, acquired at the boundary and released when the request ends. A service never constructs its own connection or engine.

Reason: a connection opened inside a service and not released leaks. Under load the pool exhausts, and the failure appears as unrelated timeouts across the whole service — rarely traced back to the one handler that opened it.

```
// ✗ a service that builds its own connection owns a leak
class LoanService {
  async settle(id: string) {
    const client = await new Pool(dbConfig).connect(); // never released on throw
  }
}

// ✓ the connection is injected and its lifetime is owned by the caller
class LoanService {
  constructor(private readonly db: DbClient) {}
  async settle(id: string) { return this.db.query(/* ... */); }
}
```

- **[NEITHER]** Pool size is configured deliberately and documented. The default is rarely right for the instance size.
- **[NEITHER]** Long-running work — a report, a batch job — does not hold a request-scoped connection. It takes its own, outside the pool used for request traffic.

13.2 Transactions

[NEITHER] Multiple writes that must succeed or fail together go in one transaction.

If a failure partway through would leave related records inconsistent — money moved but status not updated, a record created but a linked one missing — the writes go in a transaction. *A partial write is a data-integrity bug that surfaces days later as a support ticket nobody can explain.*

```
await db.transaction(async (tx) => {
  await tx.loan.update({ where: { id }, data: { status: 'SETTLED' } });
  await tx.ledger.create({ data: { loanId: id, amountMinor: toWire(payload) } });
});
```

- **[NEITHER]** Never hold a transaction open across an external API call. The connection is held hostage for the whole remote timeout and the pool drains. Do the remote call first, or record intent and reconcile.
- **[NEITHER]** A migration ships with a rollback plan, and is safe to run on a populated table. An applied migration is never edited.
- **[NEITHER]** Schema changes go through migrations only. Never hand-patch production.

13.3 Idempotency

[NEITHER] Any operation that can be retried carries an idempotency key, and a repeat returns the original result rather than performing the work again.

Clients retry. Load balancers retry. Payment providers and chains redeliver. A retried transfer is not a harmless retry.

```
// The key is supplied by the caller and stored with the result.
const existing = await idempotency.find(key);
if (existing) return existing.result;

const result = await db.transaction(async (tx) => {
  const r = await performTransfer(tx, dto);
  await tx.idempotency.create({ data: { key, result: r } }); // same transaction
  return r;
});
```

- **[NEITHER]** The key and the result are written in the same transaction as the work. Written afterwards, a crash in between leaves the work done

and the key missing.

- [NEITHER] Webhook and event handlers key on the provider's event id and tolerate duplicates.

14. Blockchain / Web3 integration

- [NEITHER] **Private keys and mnemonics never touch application code, env files, or logs.** Signing happens through the custody or KMS provider only. *Reason:* a key in an env file is a total loss event.
- [NEITHER] Every on-chain write is idempotent and keyed to an internal reference, so a retry or a redelivered event cannot double-execute.
- [NEITHER] **Chain event handlers tolerate duplicates, gaps and out-of-order delivery**, and reconcile against on-chain state rather than assuming the listener saw everything. *Reason:* missed events are a normal operating condition, not an exception.
- [NEITHER] Confirmation depth and reorg handling are explicit for each write path. **A submitted transaction is not treated as final.**
- [NEITHER] Amounts are handled as `bigint` in base units end to end. Conversion to display units happens only at the presentation boundary.
- [NEITHER] Gas estimation and transaction cost paths have failure handling and alerting. Unbounded loops and repeated calls in on-chain interaction are avoided as an explicit cost concern.
- [NEITHER] Contract addresses, chain IDs and ABIs come from validated config per environment, **never hardcoded inline.**

14.1 The chain is the source of truth

[NEITHER] **Client-supplied transaction hashes, amounts and statuses are never trusted as-is.**

The backend re-fetches the transaction from the chain and verifies hash, sender, recipient, amount and confirmation status match what the client claims — before any balance, NFT or lease state is updated.

Reason: a client can submit a hash for someone else's transaction, an unconfirmed one, or a fabricated success with no on-chain match. **The chain is the only source of truth.**

```
export async function confirmPurchase(orderId: string, hash: `0x${string}`, user: User) {
  const order = await orders.findByIdForOwner(orderId, user.id); // 8.3
  if (!order) throw NotFound('ORDER_NOT_FOUND', 'Order not found');

  const receipt = await publicClient.getTransactionReceipt({ hash });
  if (receipt.status !== 'success') throw Conflict('TX_NOT_SUCCESSFUL', 'Transaction did not succeed');

  const confirmations = await publicClient.getTransactionConfirmations({ hash });
  if (confirmations < config.minConfirmations) {
    throw Conflict('TX_NOT_CONFIRMED', 'Transaction is not yet final');
  }

  const log = decodePurchaseLog(receipt.logs, config.contracts.exchange);
  if (!log) throw Conflict('TX_NOT_FOR_THIS_CONTRACT', 'No matching event in this transaction');
  if (log.buyer.toLowerCase() !== user.walletAddress.toLowerCase()) throw Forbidden();
  if (log.listingId !== order.listingId) throw Conflict('TX_MISMATCH', 'Transaction does not match order');
  if (log.amount !== fromWire(order.amountBase)) throw Conflict('TX_AMOUNT_MISMATCH', 'Amount does not match');

  return orders.markPaid(order.id, hash); // only now does our state change
}
```

Every one of those checks exists because skipping it is exploitable.

14.2 Off-chain signatures

[NEITHER] Every off-chain signature the backend generates for on-chain use — permits, meta-transactions, relayed approvals — **includes a nonce and an expiry deadline, and the nonce is invalidated once used.**

Reason: without a nonce a valid signature can be replayed to repeat the same action. Without a deadline a signature stays usable indefinitely, including after the situation it was issued for has changed.

Appendix — ESLint starting point

Baseline existing violations rather than fixing them all at once: suppress what exists today and fail the build only on new violations.

```
// eslint.config.js (flat config)
export default [
  {
    files: ['src/**/*.ts'],
    languageOptions: { parserOptions: { project: './tsconfig.json' } },
    rules: {
      '@typescript-eslint/no-floating-promises': 'error',
      '@typescript-eslint/no-misused-promises': 'error',
      '@typescript-eslint/await-thenable': 'error',
      '@typescript-eslint/no-explicit-any': 'error',
      '@typescript-eslint/no-non-null-assertion': 'error',
      '@typescript-eslint/no-unused-vars': 'error',
      '@typescript-eslint/naming-convention': ['error',
        { selector: 'variable', modifiers: ['const'], format: ['camelCase', 'UPPER_CASE'] },
        { selector: 'typeLike', format: ['PascalCase'] } ],
    },
    'no-console': 'error',
    'no-empty': ['error', { allowEmptyCatch: false }],
    'no-sync': 'error',
    'max-lines-per-function': ['error', 75],
    'max-depth': ['error', 3],
    'sonarjs/no-commented-code': 'error',
    'no-restricted-properties': ['error', {
      object: 'process', property: 'env',
      message: 'Read config through src/config. Standard 9.',
    }],
    'no-restricted-syntax': ['error', {
      selector: "BinaryExpression[operator=/^[*/+-]$/] > Identifier[name=/[Aa]mount|[Pp]rice|[Bb]alance|[Ff]ee/]",
      message: 'Arithmetic on an amount. Use bigint minor units. Standard 12.',
    }],
  },
  { files: ['src/config/**/*.ts', 'scripts/**/*.ts'], rules: { 'no-restricted-properties': 'off', 'no-console': 'off' } },
  { files: ['**/*.test.ts'], rules: { 'max-lines-per-function': 'off' } },
];
```

The money rule is a blunt heuristic — it will produce false positives on variables that merely contain those words. Treat it as a prompt to look, not a verdict. The real check is review item 6.